

Unit 4: Cascading Style Sheets

Prepared By:
Bipin Maharjan

Learning Objectives

- Understand CSS's role in web development and its syntax.
- Apply CSS rules inline, internally, and externally.
- Master various CSS selectors, including elements, classes, IDs, and pseudo-classes.
- Grasp the concept of the box model, dimensions, and box-sizing property.
- Style typography with fonts, sizes, colors, alignment, and decorations.
- Apply diverse color formats and configure backgrounds with images and properties.
- Control element positioning using static, relative, absolute, and fixed positioning.
- Implement floating elements and handle clearing and clearfix techniques.
- Utilize display property for versatile element presentation.
- Acquire skills in creating layouts and enhancing webpage aesthetics with CSS.
- Understand viewport Meta tags for responsive design.
- Implement media queries for diverse screen sizes and responsiveness.
- Design responsive layouts using CSS for optimal viewing on various

Table of Contents

- Introduction to CSS
- CSS syntax
- Using CSS with HTML
- CSS Selectors
- CSS Comments
- CSS Properties
 - Backgrounds
 - Border, Margin
 - Padding
 - Height
 - Width
 - Color(color wheel)
- Text
- Font
- Alignment
- Line Height
- Box Model
- Working with Images
- Layout and Positioning
- Media Query
- CSS Website Layout

Introduction to CSS

What is CSS?

- Cascading Style Sheets (CSS) is a stylesheet language used to describe the presentation of an HTML document.
- While HTML is responsible for defining the structure and content of a webpage (such as headings, paragraphs, images, and links), CSS controls how those elements appear on the screen.
- In simple terms:
 - HTML = What the content is
 - CSS = How the content looks
- CSS allows developers to apply styles such as colors, fonts, spacing, borders, layouts, and positioning to HTML elements.

Why CSS is Needed?

Before CSS, web pages were designed using only HTML tags such as `<font>`, `<center>`, and `<bgcolor>`. This approach caused several problems:

- Mixing content with design
- Poor maintainability
- Repetition of styling code
- Inconsistent appearance across pages

CSS was introduced to solve these problems by separating content from presentation.

Advantages of CSS

- **Separation of Content and Design**
 - HTML handles structure; CSS handles design.
- **Consistency Across Pages**
 - One CSS file can style multiple web pages uniformly.
- **Easy Maintenance**
 - Changing one CSS file can update the appearance of an entire website.
- **Improved Page Performance**
 - Less repeated code leads to faster loading pages.
- **Responsive Design Support**
 - CSS allows web pages to adapt to different screen sizes.

Role of CSS in Web Development

- Visual design and branding
- Layout creation (columns, sections, navigation)
- User-friendly interfaces
- Responsive and mobile-friendly websites
- Accessibility improvements

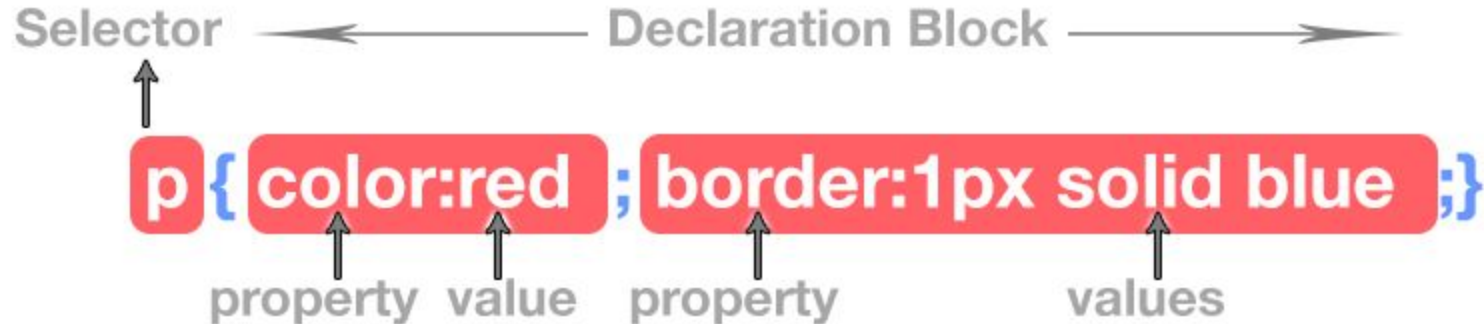
CSS Syntax

Structure of a CSS Rule

A CSS rule consists of two main parts:

- Selector
- Declaration Block

General Syntax:



Components of CSS Syntax

1. Selector

The selector specifies which HTML element(s) the CSS rule will apply to.

Examples of selectors:

- Element selector (p, h1)
- Class selector (.box)
- ID selector (#header)

Example:

```
p {  
    color: red;  
}
```

Components of CSS Syntax

2. Declaration Block

The declaration block is enclosed within curly braces { } and contains one or more CSS declarations.

3. Declaration

A declaration defines one style rule and consists of:

- Property
- Value

Each declaration must end with a semicolon (;).

Example: `color: blue;`

Components of CSS Syntax

4. Property

The property indicates what aspect of the element you want to style, such as:

- color
- font-size
- background-color
- margin

5. Value

The value specifies how the property should appear.

Example:

```
font-size: 16px;
```

Using CSS with HTML

Using CSS with HTML

CSS can be applied to HTML documents in three different ways.

- Inline CSS
- Internal or Embedded CSS
- External CSS

Inline CSS

- Involves applying styles directly to individual HTML elements using the style attribute

```
<p style="color:#009900;  
    font-size:50px;  
    font-style:italic;  
    text-align:center;">
```

Inline CSS

```
</p>
```

Inline CSS

Internal or Embedded CSS

- CSS defined within the HTML document's `<style>` element
- It applies styles to specific HTML elements
- The CSS rule set should be within the HTML file in the head section

```
<!DOCTYPE html>
<html>

<head>
  <style>
    .main {
      text-align: center;
    }

    .GFG {
      color: #009900;
      font-size: 50px;
      font-weight: bold;
    }

    .geeks {
      font-style: bold;
      font-size: 20px;
    }
  </style>
</head>

<body>
  <div class="main">
    <div class="GFG">Internal CSS</div>

    <div class="geeks">
      Implementation of Internal CSS
    </div>
  </div>
</body>

</html>
```

Internal or Embedded CSS: Example

Internal CSS
Implementation of Internal CSS

External CSS

- External CSS contains separate CSS files that contain only style properties with the help of tag attributes
- CSS property is written in a separate file with a .css extension and should be linked to the HTML document using a link tag.

External CSS: Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="main">
    <div class="GFG">External CSS </div>
    <div id="geeks">
      This shows implementation of External
      CSS
    </div>
  </div>
</body>
</html>
```

External CSS

This shows implementation of External CSS

```
body {
  background-color: powderblue;
}

.main {
  text-align: center;
}

.GFG {
  color: #009900;
  font-size: 50px;
  font-weight: bold;
}

#geeks {
  font-style: bold;
  font-size: 20px;
}
```

CSS Selectors

What is CSS Selectors?

- CSS selectors are patterns used to select HTML elements that you want to style. A selector tells the browser which elements the CSS rules should apply to.
- CSS selectors are used to "find" (or select) the HTML elements you want to style.

Types of Selectors

There are 6 types of selectors:

1. Element Selector
2. Class Selector
3. Id Selector
4. Group Selector
5. Universal Selector
6. Pseudo Classes

Element Selector

- The element selector selects all HTML elements based on the element name.

Syntax:

```
element {  
    property: value;  
}
```


Element Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
<body>
  <p>This is the first paragraph.</p>
  <p>This is the second paragraph.</p>
  <p>This is the third paragraph.</p>
</body>
</html>
```

This is the first paragraph.

This is the second paragraph.

This is the third paragraph.

Class Selector

- The class selector selects elements with a specific class attribute. It is reusable and can be applied to multiple elements.

Syntax:

```
.classname {  
    property: value;  
}
```

Class Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .important {
      color: red;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <p class="important">This is an important
paragraph.</p>
  <p>This is a normal paragraph.</p>
  <p class="important">This is another
important paragraph.</p>
</body>
</html>
```

This is an important paragraph.

This is a normal paragraph.

This is another important paragraph.

Id Selector

- The ID selector selects a single unique element. An ID must be unique within a page.

Syntax:

```
#idname {  
    property: value;  
}
```

Id Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #mainHeading {
      background-color: lightblue;
      text-align: center;
      padding: 10px;
    }
  </style>
</head>
<body>
  <h1 id="mainHeading">Welcome to My
Website</h1>
  <h1>Another Heading</h1>
</body>
</html>
```

Welcome to My Website

Another Heading

Group Selector

- The group selector applies the same styles to multiple selectors at once.

Syntax:

```
selector1, selector2 {  
    property: value;  
}
```

Group Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1, h2, p {
      color: green;
      font-family: Arial;
    }
  </style>
</head>
<body>
  <h1>Main Heading</h1>
  <h2>Sub Heading</h2>
  <p>This is a paragraph.</p>
</body>
</html>
```

Main Heading

Sub Heading

This is a paragraph.

Universal Selector

- The universal selector selects all elements on a webpage.

Syntax:

```
* {  
    property: value;  
}
```


Universal Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    * {
      margin: 10px;
      padding: 10px;
      font-family: Verdana;
    }
  </style>
</head>
<body>
  <h1>Heading</h1>
  <p>Paragraph text</p>
  <div>Div content</div>
</body>
</html>
```

Heading

Paragraph text

Div content

Pseudo-class Selector

Pseudo-classes define a special state of an element, such as hover, focus, or visited.

Common Pseudo-classes

- :hover
- :active
- :visited
- :focus

Syntax:

```
selector:pseudo-class {  
    property: value;  
}
```

Pseudo-class Selector Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    a {
      color: blue;
      text-decoration: none;
    }

    a:hover {
      color: red;
      text-decoration: underline;
    }
  </style>
</head>
<body>
  <a href="#">Hover over this link</a>
</body>
</html>
```

Hover over this link

Selector Priority Order

1. ID selector
2. Class selector
3. Element selector

Note:

- ID overrides class
- Class overrides element

Priority Order Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p {
      color: green;
    }

    .text {
      color: blue;
    }

    #unique {
      color: red;
    }
  </style>
</head>
<body>
  <p>This paragraph is green.</p>
  <p class="text">This paragraph is blue.</p>
  <p id="unique">This paragraph is red.</p>
</body>
</html>
```

This paragraph is green.

This paragraph is blue.

This paragraph is red.

CSS Comments

What are CSS Comments?

- CSS comments are used to add explanatory notes inside CSS code. These comments are ignored by the browser and do not affect the styling or output of a webpage.
- Comments are meant only for humans, not for browsers. They help in understanding, maintaining, and debugging CSS code.

Syntax of CSS Comments

- CSS uses only one type of comment syntax.

Syntax:

```
/* This is a CSS comment */
```


Comment Example

Single-Line Comment Example

```
/* This style is for paragraphs */
```

```
p {  
    color: blue;  
}
```

Multi-Line Comment Example

```
/*
```

```
This CSS section  
styles the main content  
of the webpage
```

```
*/
```

```
div {  
    padding: 20px;  
    border: 1px solid black;  
}
```

CSS Properties

Introduction to CSS Properties

CSS properties are the foundation of web design, used to style and control the behaviour of HTML elements. They define how elements look and interact on a webpage.

- Used to control layout, colors, fonts, spacing, and animations on web pages.
- It is essential for making web pages responsive and accessible across devices.
- Help maintain consistency and efficiency in web design and development.

Background Properties

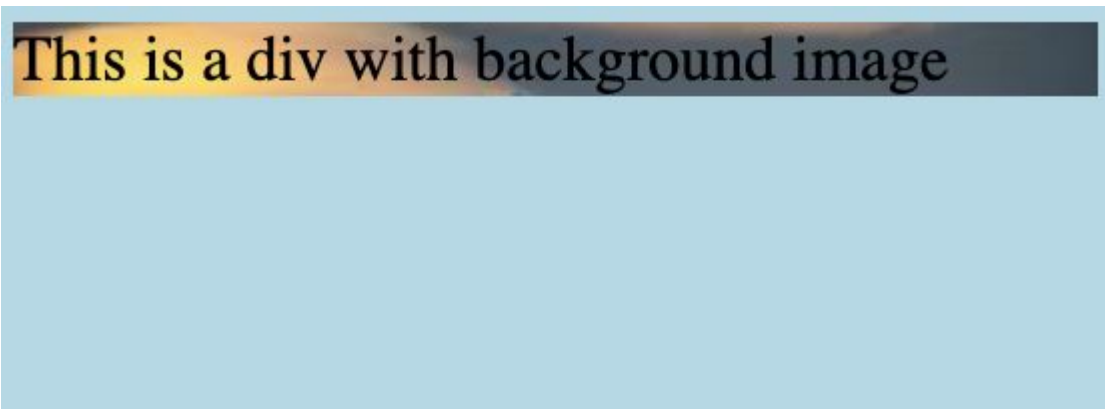
Background properties are used to control the background appearance of elements. These are commonly used for the body, div, sections, etc.

Property	Description
<code>background-color</code>	Sets background color
<code>background-image</code>	Sets background image
<code>background-repeat</code>	Controls image repetition
<code>background-position</code>	Sets image position
<code>background-size</code>	Controls image size

Background Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    background-color: lightblue;
}

div {
    background-image: url("image.jpg");
    background-repeat: no-repeat;
    background-position: center;
}
</style>
</head>
<body>
<div>This is a div with background image</div>
</body>
</html>
```



Border Property

- The border property is used to define a boundary around an element.

A border has:

- Width
- Style
- Color

Syntax

```
border: width style color;
```

Example

```
div {  
    border: 2px solid black;  
}
```

Border Property

Individual Border Properties

`border-width: 2px;`

`border-style: solid;`

`border-color: red;`

Common Border Styles

- solid
- dashed
- dotted
- double

Margin Property

Margin controls the space outside an element, creating distance between elements.

Syntax:

```
margin: value;
```

Example:

```
p {  
    margin: 20px;  
}
```

Individual Sides

```
margin-top: 10px;
```

```
margin-right: 15px;
```

```
margin-bottom: 10px;
```

```
margin-left: 15px;
```

Shorthand Example

```
margin: 10px 20px;
```


Padding Property

Padding controls the space inside an element, between content and border.

Syntax

```
padding: value;
```

Example

```
div {  
    padding: 15px;  
}
```

Individual Sides

```
padding-top: 10px;
```

```
padding-right: 20px;
```

```
padding-bottom: 10px;
```

```
padding-left: 20px;
```

Height Property

The height property sets the vertical size of an element.

Syntax

height: value;

Example

```
div {  
    height: 150px;  
    background-color: lightgray;  
}
```

Width Property

The width property sets the horizontal size of an element.

Syntax

```
width: value;
```

Example

```
div {  
    width: 300px;  
}
```

Percentage Width Example

```
div {  
    width: 50%;  
}
```

Color

- In CSS, color is used to style text, backgrounds, borders, and other visual elements to improve readability, aesthetics, and user experience.
- CSS provides multiple ways to define colors, all of which are derived from the color wheel concept.

Color Names

- CSS supports predefined color names

Example

```
p {  
    color: red;  
}
```

Common Color Names

- red
- blue
- green
- black
- white

Limitations

- Limited choices
- Less precise control

RGB Color Format

RGB stands for:

- Red
- Green
- Blue

Each value ranges from 0 to 255.

Syntax

```
color: rgb(red, green, blue);
```

Example

```
h1 {  
    color: rgb(255, 0, 0);  
}
```

This produces red.

Black and White

```
color: rgb(0, 0, 0); /* Black */  
  
color: rgb(255, 255, 255); /* White */
```

HEX Color Format

- HEX (hexadecimal) colors start with # followed by six characters.

Format:

- First two → Red
- Next two → Green
- Last two → Blue

Syntax

color: #RRGGBB;

Example

```
p {  
    color: #00ff00;  
}
```

This produces green.

Short HEX Form

```
color: #fff; /* white */
```

RGBA Color Format

- RGBA adds an alpha value for transparency.
- Alpha value ranges from 0 (transparent) to 1 (opaque).

Syntax

color: rgba(r, g, b, a);

Example

```
div {  
    background-color: rgba(255, 0, 0, 0.5);  
}
```


Color Wheel

- The color wheel is a circular diagram that shows the relationship between colors.
- It helps designers choose harmonious and visually pleasing color combinations.
- [Color wheel - color theory and calculator | Canva Colors](#)

Text

Introduction to Text Properties

- Text properties in CSS are used to control the appearance, formatting, and readability of text content on a webpage.
- These properties allow developers to modify how text looks without changing the actual content written in HTML.
- Text properties are commonly applied to:
 - Paragraphs
 - Headings
 - Links
 - Lists

text-align Property

- The text-align property controls the horizontal alignment of text inside an element.

Possible Values

- left
- right
- center
- justify

Syntax

text-align: value;

Example

```
h1 {  
  
    text-align: center;  
  
}
```

text-decoration Property

- The text-decoration property adds decorative lines to text.

Common Values

- none
- underline
- overline
- line-through

Syntax

text-decoration: value;

Example

```
a {  
    text-decoration: none;  
}
```

text-transform Property

- Controls the capitalization of text.

Values

- uppercase
- lowercase
- capitalize

Syntax

```
text-transform: value;
```

Example

```
p {  
  
    text-transform: uppercase;  
  
}
```

letter-spacing Property

- Controls the space between characters.

Syntax

```
letter-spacing: value;
```

Example

```
h1 {  
    letter-spacing: 2px;  
}
```

word-spacing Property

- Controls the space between words.

Syntax

word-spacing: value;

Example

```
p {  
    word-spacing: 5px;  
}
```


line-height Property

- The line-height property controls the vertical spacing between lines of text.

Syntax

line-height: value;

Example

```
p {  
    line-height: 1.6;  
}
```

text-indent Property

- Used to indent the first line of a text block.

Syntax

text-indent: value;

Example

```
p {  
    text-indent: 30px;  
}
```

Example

```
<style>
p {
  color: #333;
  text-align: justify;
  text-indent: 40px;
  line-height: 1.8;
  word-spacing: 4px;
}

h1 {
  text-align: center;
  letter-spacing: 3px;
  text-transform: uppercase;
}
</style>
```

```
<body>
```

```
<h1>Text Properties Example</h1>
```

```
<p>
```

CSS text properties help improve readability and appearance of textual content on web pages. Proper spacing and alignment enhance user experience.

```
</p>
```

```
</body>
```

TEXT PROPERTIES EXAMPLE

CSS text properties help improve readability and appearance of textual content on web pages. Proper spacing and alignment enhance user experience.

Font

Introduction to Font Properties

- Font properties in CSS are used to control the appearance and style of text fonts on a webpage.
- While text properties deal with alignment and decoration, font properties focus on the typeface, size, weight, and style of text.
- Fonts play a major role in:
 - Readability
 - Professional appearance
 - User experience

font-family Property

- The font-family property specifies the typeface for text. Multiple fonts are provided as a fallback list in case the browser does not support the first font.

Syntax

font-family: font1, font2, generic-family;

Example

```
p {  
    font-family: Arial, Helvetica, sans-serif;  
}
```

Generic Font Families

- serif
- sans-serif
- monospace
- cursive
- fantasy

font-size Property

- The font-size property controls the size of text.

Common Units

- px (pixels)
- em
- %

Syntax

font-size: value;

Example

```
h1 {  
    font-size: 24px;  
}
```

Relative Size Example

```
p {  
    font-size: 1.2em;  
}
```

font-weight Property

- Controls the thickness or boldness of text.

Values

- normal
- bold
- Numeric values (100–900)

Syntax

font-weight: value;

Example

```
p {  
    font-weight: bold;  
}
```

Numeric Example

```
h2 {  
    font-weight: 600;  
}
```


font-style Property

- Defines the style of the font, such as italic.

Values

- normal
- italic
- Oblique

Syntax

font-style: value;

Example

```
em {  
    font-style: italic;  
}
```

font-variant Property

- Used to display text in small capital letters.

Syntax

```
font-variant: small-caps;
```

Example

```
p {  
    font-variant: small-caps;  
}
```

Shorthand font Property

- The font property is a shorthand used to set multiple font properties in one line.

Syntax

font: style variant weight size family;

Example

```
p {  
    font: italic small-caps bold 16px Arial, sans-serif;  
}
```

Font Example

```
<style>
body {
  font-family: Verdana, sans-serif;
}

h1 {
  font-size: 28px;
  font-weight: bold;
}

p {
  font-size: 16px;
  font-style: italic;
}
</style>
```

```
<body>
```

```
<h1>Font Properties Example</h1>
```

```
<p>
```

CSS font properties allow control over typeface, size, and style, improving

readability and visual appeal.

```
</p>
```

```
</body>
```

Font Properties Example

CSS font properties allow control over typeface, size, and style, improving readability and visual appeal.

Alignment

Introduction to Alignment

- Alignment in CSS refers to the way content and elements are positioned or aligned within their parent container.
- Proper alignment improves readability, structure, and visual balance of a webpage.
- Mainly Involves:
 - Text alignment
 - Horizontal alignment of elements
 - Centering elements

Text Alignment (text-align)

- The text-align property is used to align text content horizontally inside an element.

Values

- left (default)
- right
- center
- justify

Syntax

text-align: value;

Example

```
p {  
  
    text-align: justify;  
  
}
```

This paragraph is justified. The text is aligned evenly on both the left and right sides, improving readability for long content.

Center Alignment of Block Elements (Using Margin)

- Block-level elements (like div) can be centered horizontally using:

`margin: auto;`

Syntax

`margin-left: auto;`

`margin-right: auto;`

Example

```
.box {  
    width: 300px;  
    margin: auto;  
    background-color: lightgray;  
}
```


Block Element Example

```
<!DOCTYPE html>
<html>
<head>
<style>
.box {
  width: 300px;
  margin: auto;
  padding: 15px;
  background-color: lightgray;
}
</style>
</head>
```

```
<body>

<div class="box">This box is horizontally
centered.</div>

</body>
</html>
```

This box is horizontally centered.

Vertical Alignment (vertical-align)

- The vertical-align property aligns inline or table-cell elements vertically relative to surrounding content.

Common Values

- top
- middle
- bottom
- baseline

Syntax

vertical-align: value;

Example

```
img {  
    vertical-align: middle;  
}
```

Vertical Alignment Example

```
<style>
```

```
img {
```

```
width: 40px;
```

```
height: 40px;
```

```
vertical-align: middle;
```

```
}
```

```
</style>
```

```
<h2>Vertical Alignment Demo</h2>
```

```
<p>
```

```

```

This text is aligned with the middle of the image.

```
</p>
```

Vertical Alignment Demo



This text is aligned with the middle of the image.

Aligning Images

Images are inline elements by default and can be aligned using:

- `text-align` (inside container)
- `margin: auto` (when converted to block)

Example (Center Image)

```
img {  
    display: block;  
    margin: auto;  
}
```

Alignment Image Example

```
<style>
```

```
img {
```

```
  display: block;
```

```
  margin: auto;
```

```
  width: 200px;
```

```
}
```

```
</style>
```

```
<body>
```

```
<h2>Centered Image</h2>
```

```

```

```
</body>
```

Centered Image



Line Height

Introduction to Line Height

- Line height in CSS controls the vertical spacing between lines of text within an element.
- It directly affects readability, especially for paragraphs and long textual content

Syntax

line-height: value;

Accepted Values

- Normal (default)
- Number (unitless)
- Length (px, em, rem)
- Percentage

Normal (Default Value)

Browser automatically sets spacing based on font size.

Example

```
p {  
    line-height: normal;  
}
```


Unitless Number (Recommended)

- Multiplies the font size

Example

```
p {  
    font-size: 16px;  
    line-height: 1.5;  
}
```

This means:

$16\text{px} \times 1.5 = 24\text{px}$ line height

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  font-size: 16px;
  line-height: 1.5;
}
</style>
</head>
<body>
```

```
<p>
This paragraph uses unitless line height. It scales automatically
with font size and is recommended for responsive design.
</p>
```

```
</body>
</html>
```

Length Values (px, em)

- Defines exact spacing between lines.

Example

```
p {  
    line-height: 30px;  
}
```

Percentage Value

- Percentage of the font size.

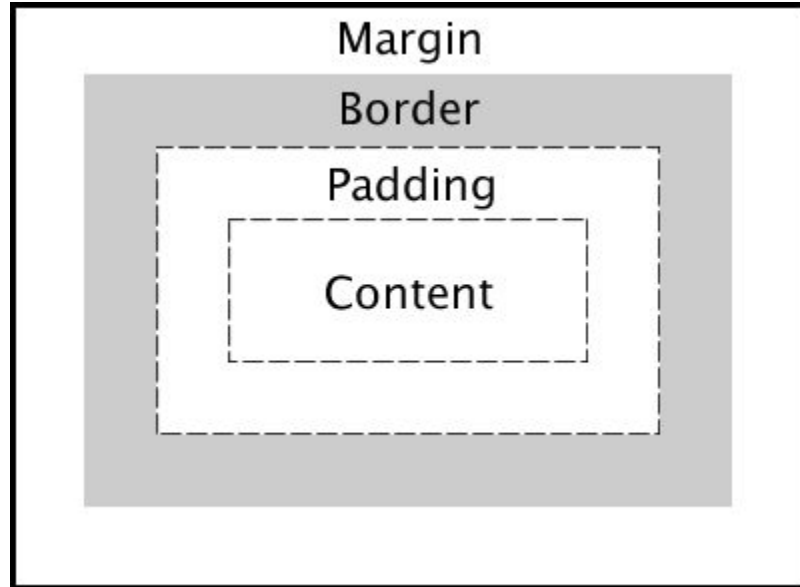
Example

```
p {  
    line-height: 150%;  
}
```

Box Model

Introduction to the Box Model

The CSS Box Model is a fundamental concept that explains how every HTML element is treated as a rectangular box by the browser.



Content

- The actual area where text, images, or other content appears
- Its size is controlled by width and height

Syntax

width: value;

height: value;

Example

```
div {  
    width: 200px;  
    height: 100px;  
}
```

Padding

- Space between content and border
- Padding increases the inner space of the element
- Padding is transparent

Syntax

`padding: value;`

Or, Individually

`padding-top`

`padding-right`

`padding-bottom`

`padding-left`

Example

```
div {  
    padding: 20px;  
}
```


Border

- Border surrounds padding and content
- Has width, style, and color

Syntax

`border: width style color;`

Example

```
div {  
    border: 2px solid black;  
}
```

Margin

- Space outside the border
- Creates distance between elements
- Margin is transparent

Syntax

margin: value;

Example

```
div {  
    margin: 20px;  
}
```

Box Example

```
<style>
.box1 {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: content-box;
  background-color: lightgray;
}

.box2 {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
  background-color: lightblue;
}
</style>
```

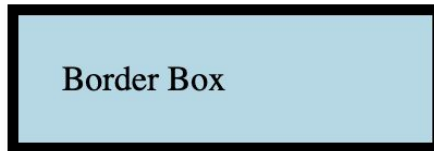
```
<body>

<div class="box1">Content Box</div>

<br>

<div class="box2">Border Box</div>

</body>
```



Working with Images

Introduction

- Working with images in CSS involves controlling how images are:
 - Displayed
 - Sized
 - Positioned
 - Decorated

Using Width and Height

Using Width and Height

Syntax

```
img {  
    width: value;  
    height: value;  
}
```

Adding Border, Padding, and Margin to Images

- Images follow the box model, so all box properties apply.

Example

```
img {  
    border: 2px solid black;  
    padding: 10px;  
    margin: 20px;  
}
```

Example

```
<style>
```

```
img {
```

```
  width: 150px;
```

```
  border: 3px solid black;
```

```
  padding: 10px;
```

```
  margin: 20px;
```

```
}
```

```
</style>
```

```
<body>
```

```

```

```
</body>
```



Rounded Images

You can use the `border-radius` property to create rounded images:

Example

```
img {  
    border-radius: 8px;  
}
```



Making Images Responsive

- Responsive images adapt to different screen sizes.

Key Property

`max-width: 100%;`

`height: auto;`

Example

```
<style>
```

```
  img {
```

```
    max-width: 100%;
```

```
    height: auto;
```

```
  }
```

```
</style>
```

```
<body>
```

```
<div style="width: 300px;">
```

```
  
```

```
</div>
```

```
</body>
```

Background Images

Syntax

`background-image: url("image.jpg");`

Property	Description
background-repeat	repeat / no-repeat
background-size	cover / contain
background-position	center, top
background-attachment	scroll / fixed

Example

```
<style>
```

```
  .box {
```

```
    width: 300px;
```

```
    height: 200px;
```

```
    background-image: url("image.jpg");
```

```
    background-size: cover;
```

```
    background-position: center;
```

```
    border: 2px solid black;
```

```
  }
```

```
</style>
```

```
<body>
```

```
<div class="box"></div>
```

```
</body>
```



Image Opacity

- Controls transparency of image.

Syntax

opacity: value;

Example

```
img {  
    opacity: 0.7;  
}
```

Note

- Opacity value ranges from 0 to 1.

Layout and Positioning

Introduction to Layout and Positioning

Layout and positioning in CSS determine where elements appear on a webpage and how they are arranged relative to each other.

Normal Document Flow

By default:

- Block elements appear top to bottom
 - A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.
 - Commonly used block elements are: `<p>` and `<div>`.
- Inline elements appear left to right
 - An inline element does not start on a new line.
 - An inline element only takes up as much width as necessary.
 - Commonly used inline elements are: `<span>`, `<a>`, `<img>`

This is called the normal flow.

CSS position Property

Syntax

`position: value;`

Position Values

- static
- relative
- absolute
- fixed

position: static (Default)

- Default position of all elements
- Not affected by top, left, right, bottom

Example

```
div {  
    position: static;  
}
```

position: static Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .box {
      position: static;
      border: 2px solid black;
    }
  </style>
</head>
<body>

<div class="box">Static Position</div>

</body>
</html>
```



Static Position

position: relative

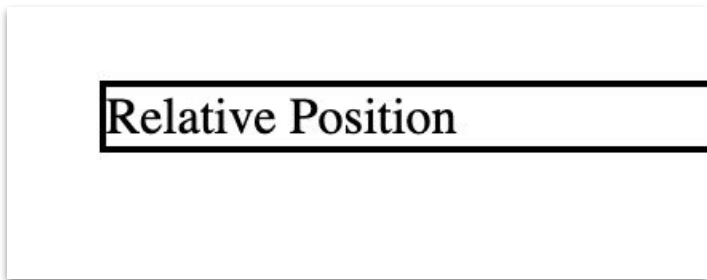
- Positioned relative to its normal position
- Original space is preserved
- Used as reference for absolute positioning

Syntax

position: relative;

top: value;

left: value;



```
<style>
.box {
  position: relative;
  top: 20px;
  left: 30px;
  border: 2px solid black;
}
</style>
```

```
<body>
<div class="box">Relative Position</div>
</body>
```

position: absolute

- Positioned relative to nearest positioned ancestor
- Removed from normal flow
- If no positioned ancestor, relative to <body>

Syntax

position: absolute;

top: value;

left: value;

position: absolute Example

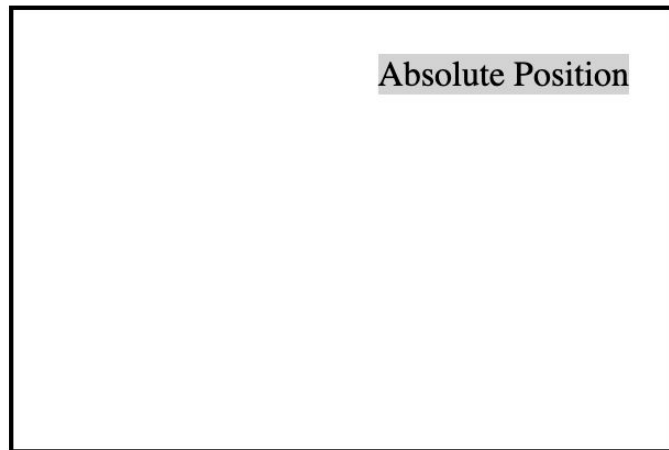
```
<style>
.container {
  position: relative;
  width: 300px;
  height: 200px;
  border: 2px solid black;
}

.box {
  position: absolute;
  top: 20px;
  right: 20px;
  background-color: lightgray;
}
</style>
```

```
<div class="container">

  <div class="box">Absolute Position</div>

</div>
```



position: fixed

- Positioned relative to the viewport
- Stays fixed even on scroll
- Used for navigation bars, buttons

Syntax

`position: fixed;`

`top: value;`

`right: value;`

position: fixed Example

```
<style>
.fixed-box {
  position: fixed;
  bottom: 20px;
  right: 20px;
  background-color: black;
  color: white;
  padding: 10px;
}
</style>
```

<p>Scroll down to see fixed position effect.</p>

<div class="fixed-box">

Fixed Button

</div>

<div style="height: 1000px;"></div>

Scroll down to see fixed position effect.

Z-Index (Layering)

- Controls the stacking order of positioned elements.
- Higher z-index appears on top.

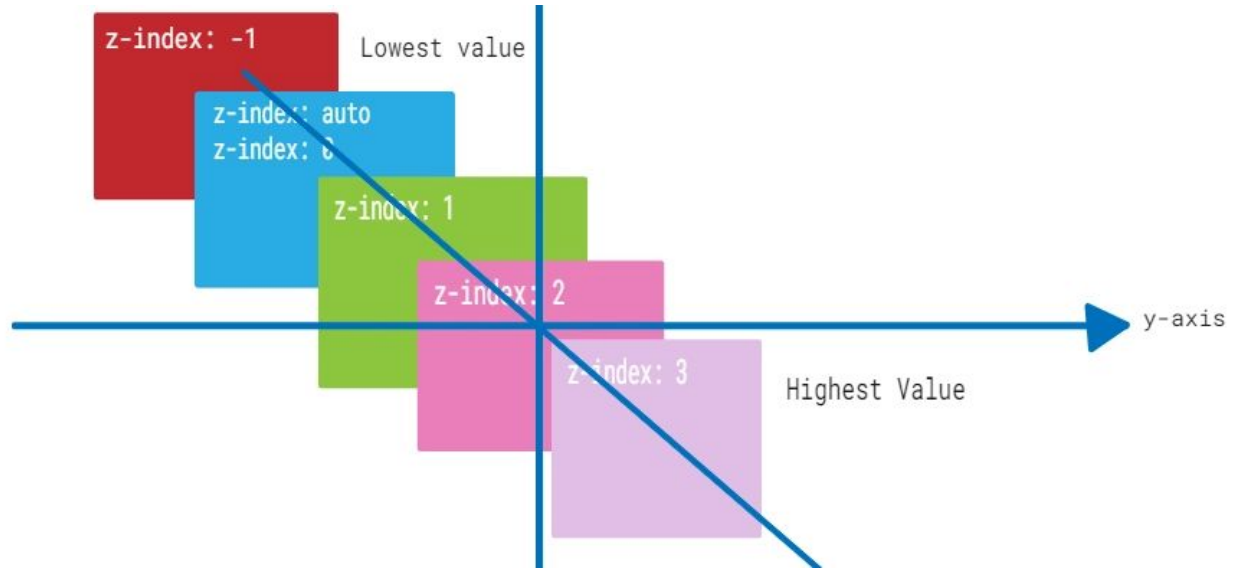
Syntax

`z-index: number;`

Example

```
.box1 { z-index: 1; }
```

```
.box2 { z-index: 2; }
```



Media Query

Introduction to Media Queries

- A media query is a CSS technique used to apply different styles for different screen sizes and devices.
- Media queries are the foundation of responsive web design, allowing a webpage to adapt to:
 - Mobile phones
 - Tablets
 - Laptops
 - Large screens

Syntax

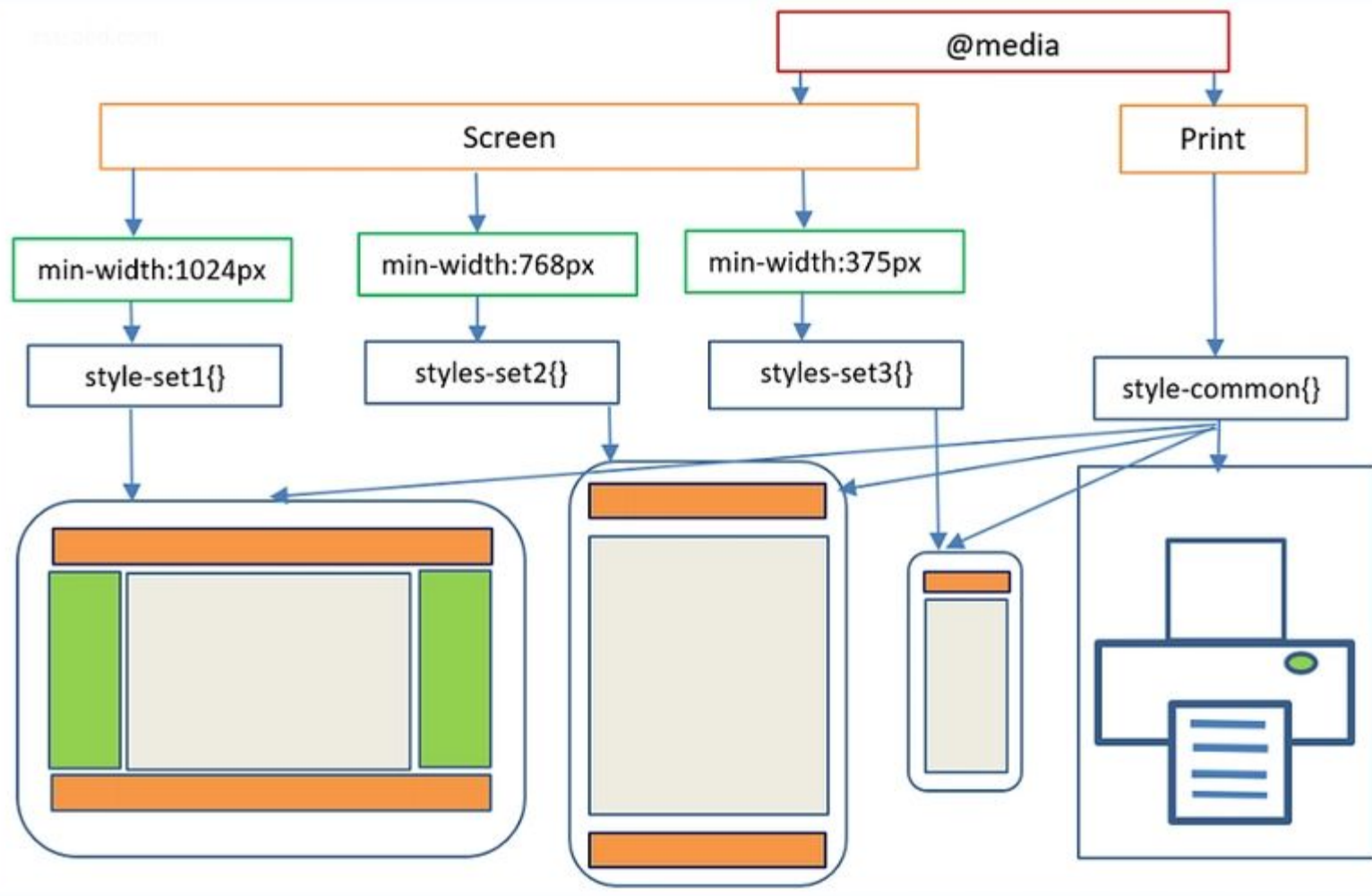
```
@media media-type and (condition) {  
    CSS rules;  
}
```

Common Media Types

- screen (most common)
- print
- all

Media Features (Conditions)

Feature	Description
max-width	Applies up to a width
min-width	Applies from a width
width	Exact width



Change background color based on screen width

```
<head>
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
<style>
body {
    background-color: lightblue;
}

/* Mobile devices */
@media screen and (max-width: 600px) {
    body {
        background-color: lightgreen;
    }
}
</style>
</head>
```

Resize the browser window

Background color changes on screen size.

```
<body>

<h2>Resize the browser window</h2>

<p>Background color changes on screen
size.</p>

</body>
```

Resize the browser window

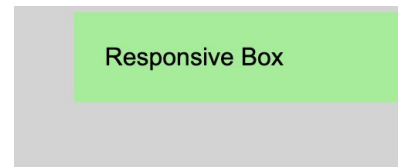
Background color changes on screen size.

Example

```
<style>
  body {
    font-family: Arial;
    background-color: lightgray;
  }
  .box {
    width: 80%;
    margin: auto;
    padding: 20px;
    background-color: white;
  }
  /* Mobile */
  @media screen and (max-width: 600px) {
    .box {
      background-color: lightyellow;
    }
  }
  /* Tablet */
  @media screen and (min-width: 601px) and
(max-width: 900px) {
    .box {
      background-color: lightgreen;
    }
  }
}
```

```
/* Desktop */
@media screen and (min-width: 901px) {
  .box {
    background-color: lightblue;
  }
}
</style>
```

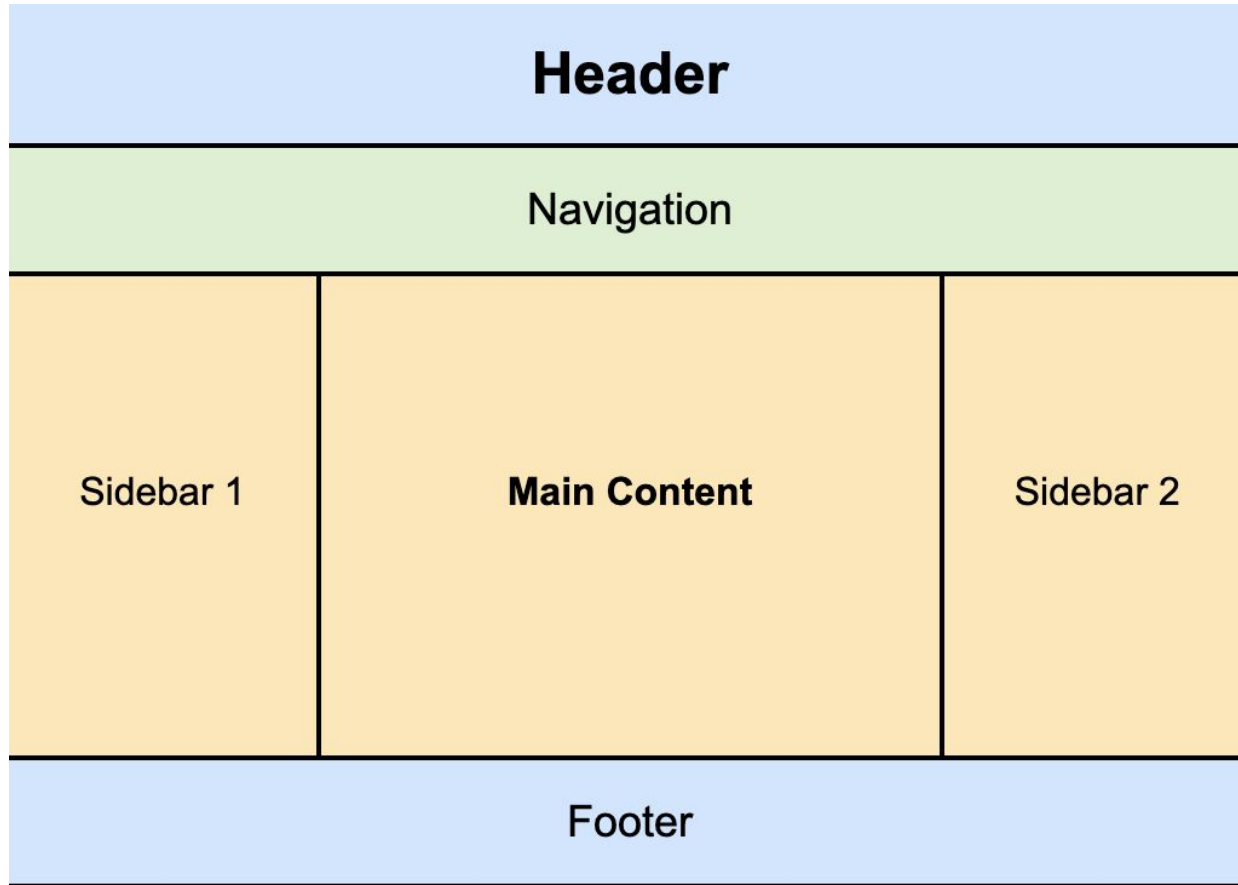
```
<body>
  <div class="box">
    Responsive Box
  </div>
</body>
```



CSS Website Layout

Introduction to Website Layout

- A website layout refers to how different sections of a webpage are arranged and structured.
- In basic CSS, layouts are created using:
 - width and height
 - float
 - clear
 - display
 - position
 - Media queries for responsiveness



Basic Structure of a Website

HTML Structure

<header>Header</header>

<nav>Navigation</nav>

<main class="container">

 <aside>Sidebar 1</aside>

 <section>Main Content</section>

 <aside>Sidebar 2</aside>

</main>

<footer>Footer</footer>

CSS Structure

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
  
header, nav, footer {  
  text-align: center;  
  padding: 15px;  
  border: 1px solid black;  
}  
  
header {  
  background-color: #cfe5ff;  
  font-size: 24px;  
  font-weight: bold;  
}  
  
nav {  
  background-color: #d9f0d0;  
  font-size: 18px;  
}
```

```
.container {  
  width: 100%;  
  margin: auto;  
}  
  
aside, section {  
  float: left;  
  height: 200px;  
  background-color: #ffe6b3;  
  text-align: center;  
  padding-top: 80px;  
  border: 1px solid black;  
  box-sizing: border-box;  
}  
  
aside {  
  width: 25%;  
}  
  
section {  
  width: 50%;  
  font-weight: bold;  
}  
  
footer {  
  clear: both;  
  background-color: #cfe5ff;  
  margin-top: 10px;  
  font-size: 18px;  
}
```

Layout Technique Used

- Float-based layout
 - Uses float to align sections horizontally
 - Requires clear to avoid overlap
- float: left for columns
- clear: both for footer

Any Questions?